Software Requirement Specification

Course Name: Project

Course no. SE 801

Supervised By

Mohd. Zulfiquar Hafiz

Professor

Institute of Information Technology

University of Dhaka

Submitted by

Sharif Mohammad Abdullah

Class Roll- 1211

Supervisor's Signature

1. Introduction **2**
1.1 Motivation 2
1.2 Scope 3
1.3 Purpose of Document 3
1.4 Project Timeline 3
2. Project Description **3**
2.1 Quality Function Deployment 5
2.1.1 Normal Requirements: 5
2.1.2 Expected Requirements: 5
2.1.3 Exciting Requirements: 5
2.2 Usage Scenario: 5
Process Initiation 5
Code Extractor and Parser 6
Detection of Vulnerabilities 6
Code Preprocessing 6
Query Parsing and Retrieve Relevant Entries: 7
Generate Augmented Response: 7
3. Scenario Based Modeling **7**
3.1 Use Case Diagram 7
3.1.1 Level-0 7
3.1.2 Level - 1 7
Preprocessor 7
Vulnerability Predictor 7
Report Generator 8
Fix Recommender 8
4. Class-Based Modeling **8**
4.1 General Classification 8
4.2 CRC(Class-Responsibility-Collaboration) Cards 10
CodePreProcessor 10
MetricsExtractor 10
VulnerabilityPredictor 10
LLMProcessor 10
ResponseGenerator 10
ReportGenerator 11
CRC Diagram 11
5. Architectural Design **11**
5.1 Architectural Context Diagram 11
5.2 Defining Archetypes 11
Layered Architecture 11

Pipe and Filter Architecture 12
Model-View-Controller (MVC) 13
5.3 Top level components 15
5.4 Instantiation of the Architecture 15
Component: PreProcessor 15
Component: PredictionResponse 16
Component: Report 16
6. Preliminary Test Plan **16**
6.1 High Level Description of Testing Goals 16
6.2 Test Cases 16
10. Conclusion **21**

Letter of Transmittal

01 December 2024
BSSE 4th Year Exam Committee
Institute of Information Technology
University of Dhaka

Subject: Software Requirement Specification of “VulScan”.

Sir,
With due respect, I am pleased to submit the Software Requirement Specification of Software Project Lab 3 on “VulScan”. I have tried my best to deliver a good document that will help people understand the project. However, it might lack perfection. So, I, therefore, hope that you would be kind enough to accept my report and oblige thereby.

Sincerely yours,
Sharif Mohammad Abdullah
Roll: BSSE 1211
Institute of Information Technology
University of Dhaka

Acknowledgment

I am grateful to the Institute of Information Technology for giving me such a tremendous opportunity to work on “VulScan”. I would like to convey my tremendous gratitude to my supervisor, Mohd. Zulfiquar Hafiz, Professor, Institute of Information Technology, University of Dhaka, for providing me guidelines regarding the preparation process of this analysis document.. Lastly, I would like to thank fellow classmates as their ideas enhanced mine and their helpful attitude significantly implicated the progress of this project.

Abstract

This document contains the software requirements and specification, architectural design and test plan of “VulScan” which is a static analyzer tool. This tool will help software developers detect vulnerabilities in their C/C++ source code with the help of the latest technologies. With this, developers can greatly ease their work of finding vulnerabilities.

Software Requirement Specification (SRS): VulScan

1. Introduction

In today's digital landscape, security vulnerabilities in software are a significant concern, especially in languages like C and C++, which are commonly used in critical systems. Identifying vulnerabilities early in the development cycle can reduce risks, prevent attacks, and save costs. Traditional static code analysis tools often produce faulty results or are limited in other ways.

1.1 Motivation

The motivation behind DiverseVul stems from the fact that C/C++ code is used for projects that are the underlying base of most of the softwares we use today. But there are significant risks associated with these codes as some of them were written not considering future threats. It is near impossible for humans to review and identify all such risky codes. As deep learning algorithms are being used to solve problems at an increasing rate each day, we explore the opportunity to use such models to detect vulnerabilities as well and predict potential vulnerabilities before they lead to security issues in production.

The project creates a simple web UI with deep learning models, particularly TabNet and LLMs, to predict vulnerabilities in C/C++ code, offering a more efficient, accurate, and scalable solution.

1.2 Scope

After conducting a one-to-one interview with the stakeholder, we found the following specifications to be within the scope of this project:

1. Create a web app for vulnerability prediction
2. Create a simple, easy to use, intuitive UI for the web app
3. Find vulnerabilities in a C function
4. Mark the vulnerabilities and identify them as per their specific categories
5. Generate report based on the identifications

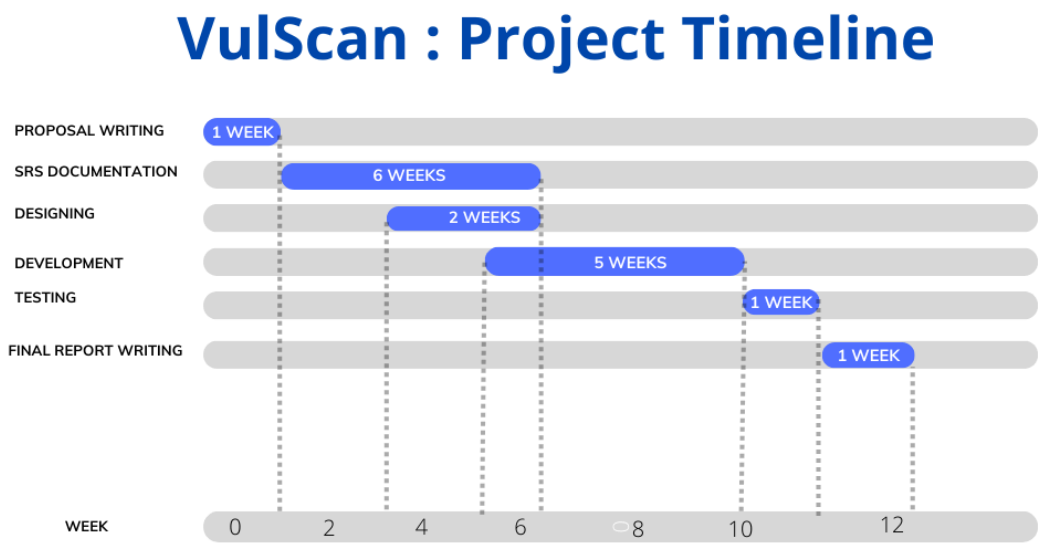
1.3 Purpose of Document

The purposes of this document are:

1. Identify and analyze the requirements
2. Design the architecture
3. Design the test plan
4. Describe methodology
5. Reduce the development effort
6. Improve understanding

1.4 Project Timeline

The estimated timeline for the whole project is such:



2. Project Description

We present the quality function deployment of VulScan in the following sections. The identification of our Quality Function Deployment (QFD) began with an in-depth review of relevant research literature, notably the paper authored by Max and Sejfia [1], together with interviews conducted with stakeholders. Our stakeholders were software developers themselves and the CEO of a software company. From the interviews and the literature reviews, we collected the necessary information for QFD.

2.1 Quality Function Deployment

Quality Function Deployment (QFD) is a technique that translates the needs of the customer into technical requirements for the software. The following requirements have been identified for this project:

2.1.1 Normal Requirements:

1. The tool should have a easy and intuitive user interface for ease of use
2. The tool should be able to provide a report identifying whether the functions used in the project are vulnerable or not
3. It should have reasonable processing times for analysis, ensuring minimal impact on user experience.
4. The tool will look through all files and no files will be ignored in the search.

2.1.2 Expected Requirements:

1. The classifiers should demonstrate a high level of accuracy in identifying vulnerable codes.
2. The report should be in such a format that an average user can understand it too.
3. The tool must not make any unnecessary changes to source code
4. The tool has to be operable in different browsers and OS

2.1.3 Exciting Requirements:

1. The tool should have a channel for users to provide feedback on false positives and false negatives.
2. In addition to basic reports, VulScan will offer detailed reports with insights into why a code has been marked as vulnerable and what to do to fix it

2.2 Usage Scenario

The software will be a web app with the following usage scenario:

Process Initiation

The tool's data analyzer will use a source code repository to create a model from the following metrics existing in the repository: availability impact,access complexity,confidentiality impact,integrity impact,authentication required. A deep tabular data analysis will be done to create a model that will learn the patterns usually found in a vulnerable code with the help of these metrics to predict vulnerability.

Code Extractor and Parser

Functions are extracted from the zip file. Then, an LLM agent with a rag pipeline is used to extract key features from all of the functions found form the extraction process. The rag system gives the system a knowledge base to work with, instead of blindly predicting features.

Detection of Vulnerabilities

VulScan will evaluate the vulnerable functions using a deep learning classification method that will use collected examples from famous repositories (such as linux) as a dataset containing functions, their vulnerability metrics and their fixed version to suggest if the user uploaded functions are vulnerable or not. If a function is deemed to be suspicious, further analysis will be done to suggest the type of vulnerability and what can be done to fix it, which are extra features.

Code Preprocessing

The tool parses the CSV file containing the knowledge base. Each row represents a document or data entry that the pipeline will retrieve from. Then it converts the CSV data into chunks and encoding text. Then it uses an embedding to represent the data. Finally it stores the vectorized data in a searchable format in a vector database. Each vector is linked to its corresponding original text.

Query Parsing and Retrieve Relevant Entries:

After accepting user input and converting the query into a vector using the same embedding model, the tool will use the vector index to find the most similar entries to the current query which is a C function and retrieve these entries for augmentation.

Generate Augmented Response:

By combining the retrieved knowledge with the query to construct a prompt for the LLM, the tool will pass the prompt to generate a contextually enriched response. Finally it will return the metrics for the input code.

3. Scenario Based Modeling

Scenario-based modeling is an approach used in software development and design to create models that depict how a system behaves or reacts in various real-world situations or scenarios. These scenarios are typically based on user interactions, events, or conditions and help developers understand and visualize how the software will function in different contexts. By simulating these scenarios, developers can better identify potential issues, make informed design decisions, and ensure that the software meets user requirements across different usage contexts.

3.1 Use Case Diagram

A use case diagram is a visual representation in software engineering that illustrates how a system interacts with its external entities or actors to achieve specific goals or tasks. It shows the relationships between use cases (representing system functionalities) and actors (representing external users or systems)

3.1.1 Level-0

Primary actors: User
Secondary actors: Code Repository

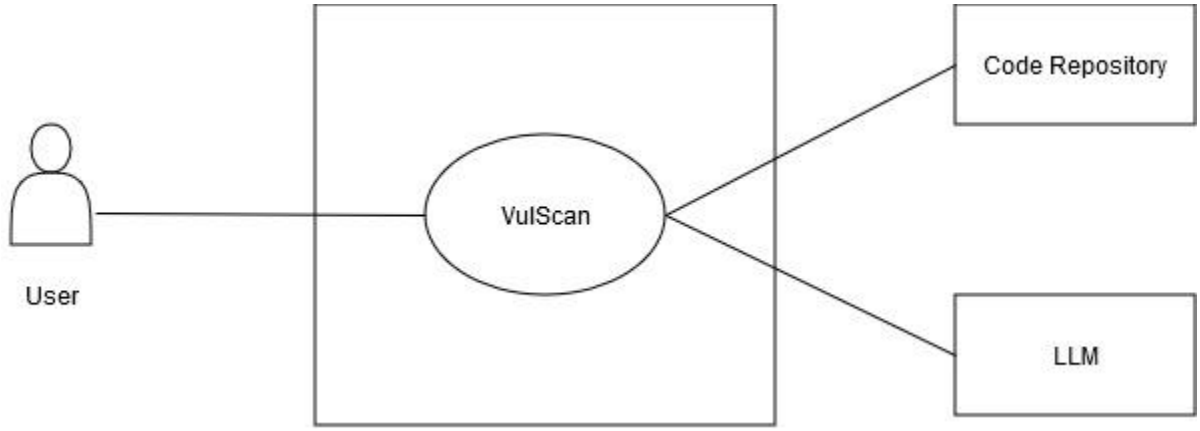


Figure: Use Case level 0

Goal in Context: The diagram shown above represents the full tool

3.1.2 Level - 1

Primary Actor: User
Secondary Actor: npm
Goal in Context: The diagram shows the modules of VulScan as shown below:

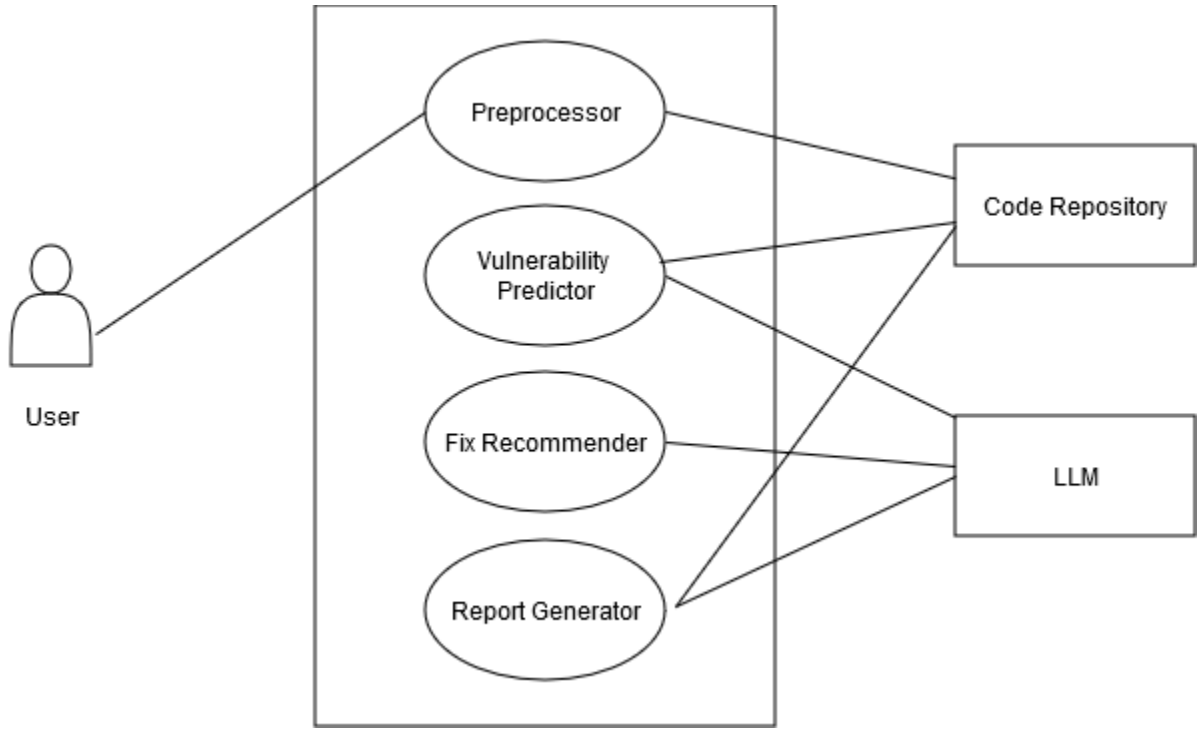


Figure: UCD Level 1

Preprocessor

Preprocessor module is responsible for adjusting the dataset into sizable and manageable parts so that it can be used for analysis. This is also responsible for tokenization, stop word removable and vectorization and embedding of the code snippets.

Vulnerability Predictor

The vulnerability predictor module is the core of the security mechanism. It employs deep learning classifiers trained on known examples of malicious and benign codes to assess the vulnerability metrics in real-time. If a code is flagged as potentially malicious, this module triggers further analysis to confirm its status.

Report Generator

The Report Generator module is responsible for creating comprehensive analysis reports for flagged codes. When the tool detects a suspicious code, it generates a detailed report explaining the specific reasons behind the flagging. This module ensures that users have access to clear and informative information about the package's potential security risks.

Fix Recommender

This module focuses on suggesting necessary changes if necessary to fix vulnerabilities of codes. It uses LLMs with a rag pipeline to suggest these changes.

4. Class-Based Modeling

Class-Based Modeling Concept

Class-based modeling identifies classes, attributes, and relationships that the system will use. It represents the object. The system manipulates the operations. The elements of the class-based model consist of classes and objects, attributes, operations, and class – responsibility - collaborator (CRC) models. Classes are determined using underlining each noun or noun clause and entering it into the simple table. Attributes are the set of data objects that are defining a complete class within the context of the problem. The operations define the behavior of an object.

4.1 General Classification

After identifying 56 nouns from the scenario, I have filtered nouns belonging to the solution domain using General Classification (External entities, Things, Events, Roles, Organizational units, Places, and Structures). Nouns selected as a potential class were filtered using Selection Criteria (Retained information, Needed services, Multiple attributes, Common attributes, Common operations, and Essential requirements). After performing analysis on potential classes, I have found the following analysis classes

1. **CodePreProcessor**: Analyzes source code to
2. **MetricsExtractor**: Extracts the various impact and complexity metrics.
3. **VulnerabilityPredictor**: Predicts vulnerability status of a function.
4. **LLMProcessor**: Handles the use of the LLM and RAG pipeline for analysis.
5. **ResponseGenerator**: Combines retrieved data and user input to generate results.
6. **ReportGenerator**: Generates reports as PDFs based on findings.

4.2 CRC(Class-Responsibility-Collaboration) Cards

Class-responsibility-collaboration (CRC) cards are a brainstorming tool used in the design of object-oriented software. CRC cards are a great tool by which multiple alternative interactions may be quickly devised, as they avoid a great deal of drawing and erasing. CRC cards are frequently employed during the design phase of system and software development to transition use-case descriptions into class diagrams, allowing a smoother transition with a greater overview and permitting developers to implement solutions with low binding and high cohesion.

The CRC cards produced for VulScan are:

CodePreProcessor	
Attribute	Method
sourceCode, functionsExtracted, preprocessedCode	extractFunctions() cleanCode()
Responsibility	Collaborators
1. Analyze the source code for relevant functions. 2. Prepare the source code for further processing.	MetricsExtractor,VulnerabilityPredictor

MetricsExtractor	
Attribute	Method
functionCode, metrics,	extractMetrics()
Responsibility	Collaborators

1. Extract and calculate relevant metrics for each function. 2. Provide metrics as input for vulnerability prediction.	CodePreProcessor, VulnerabilityPredictor

VulnerabilityPredictor	
Attribute	Method
functionCode, metrics, response	predictVulnerability(), classifyVulnerability(),
Responsibility	Collaborators
1. Analyze code and metrics to predict vulnerability status. 2. Provide detailed insights into the type and severity of vulnerabilities.	MetricsExtractor, FeedbackProcessor

LLMProcessor	
Attribute	Method
query, knowledgeBase,	queryKnowledgeBase(), generateAugmentedResponse(),
Responsibility	Collaborators
1. Use RAG (Retrieval-Augmented Generation) pipelines for analyzing functions. 2. Enhance predictions with enriched contextual knowledge.	ResponseGenerator

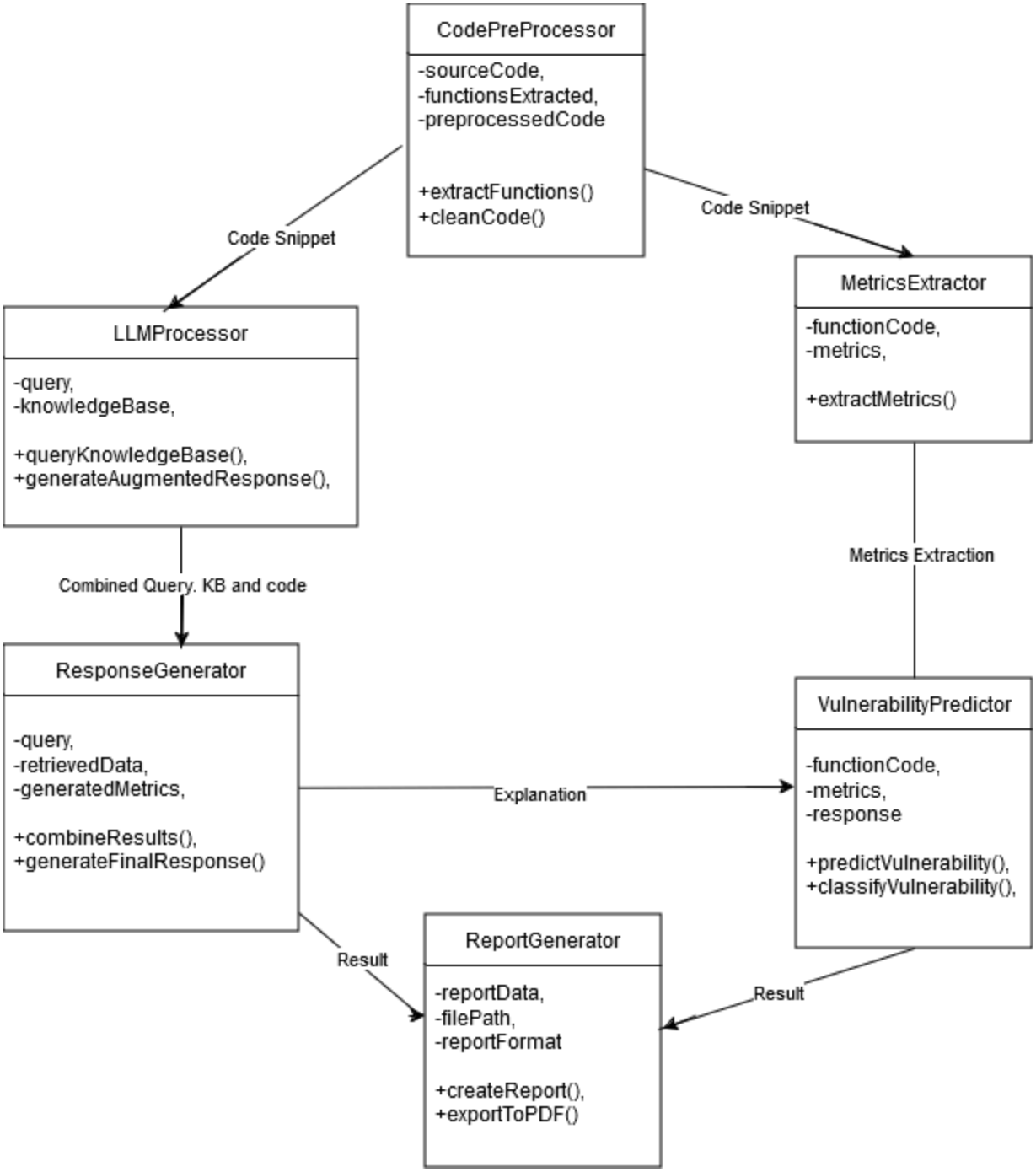
ResponseGenerator	
Attribute	Method
query,	combineResults(),

retrievedData, generatedMetrics,	generateFinalResponse()
Responsibility	Collaborators
1. Generate a coherent response for user queries. 2. Use processed data to provide predictions	LLMProcessor,ReportGenerator

ReportGenerator	
Attribute	Method
reportData, filePath, reportFormat	createReport(), exportToPDF()
Responsibility	Collaborators
1. Prepare comprehensive reports based on system findings. 2. Export findings in a user-friendly format for further use.	ResponseGenerator

CRC Diagram

The CRC diagram with all of the analysis classes can be summarised as follows



5. Architectural Design

This chapter describes the architectural context diagram, archetypes and top level components for the tool.

5.1 Architectural Context Diagram

The architectural context diagram for the tool is given below-

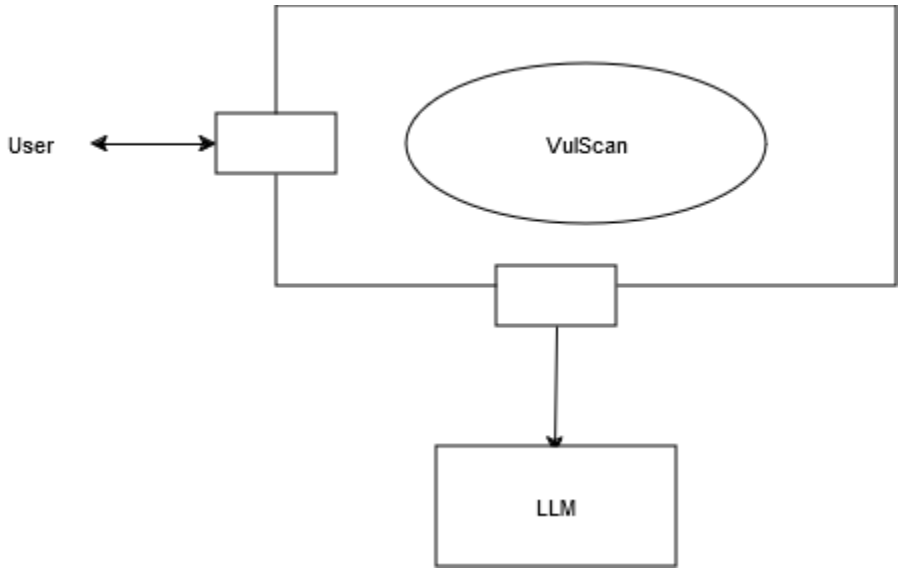


Figure 9: Architectural Context Diagram

All softwares engineers are eligible users of the tool. They are the producers and also consumers of the information the tool provides. The subordinate system is LLM. It provides necessary processing to complete target system functionality.

5.2 Defining Archetypes

The VulScan system uses the following archetypes:

1. Layered Architecture
2. Pipe and filter
3. Model-View-Controller (MVC)

Layered Architecture

Layered architecture is a software design pattern that organizes a system's components into distinct layers, with each layer serving a specific set of functionalities and having well-defined interfaces for communication.

It is commonly used in various applications, from web development to enterprise software, due to its clear separation of concerns and ease of maintenance.

Presentation Layer: This layer gets and presents data from and to the system from the user.

Business Logic Layer: Contains the main business logic of our system.

Utility Layer: It contains miscellaneous functions that are used by the business logic layer. It includes library functions, frameworks and well-established algorithms.

Persistence Layer: It contains interfaces to document access objects.

Data Layer: To store the data in storage.

Pipe and Filter Architecture

The Pipe and Filter architecture is a software design pattern that organizes a system as a series of processing elements (filters) connected by communication channels (pipes). It is particularly useful for designing applications that involve processing streams of data in a linear and sequential manner. Each filter performs a specific task on the incoming data and passes the results to the next filter through the pipes, creating a data flow pipeline.

1. **Filter:** All processes that contain a significant general algorithm that can be reused in other tasks. The filters can be re-ordered to solve different problems.
2. **Pipe:** Specific tasks that are required to communicate between the filters.

Model-View-Controller (MVC)

The Model-View-Controller (MVC) framework is an architectural/design pattern that separates an application into three main logical components Model, View, and Controller. Each architectural component is built to handle specific development aspects of an application. It isolates the business logic and presentation layer from each other.

Model: The Model component corresponds to all the data-related logic that the user works with. This can represent either the data that is being transferred between the View and Controller components or any other business logic-related data.

View: The View component is used for all the UI logic of the application. It generates a user interface for the user. Views are created by the data which is collected by the model component but these data aren't taken directly but through the controller.

Controller: The Model component corresponds to all the data-related logic that the user works with. This can represent either the data that is being transferred between the View and Controller components or any other business logic-related data.

5.3 Top level components

The overall architecture with top level components are illustrated below. Note that the top level components show a very high level view of our system. Each component has independent responsibilities.

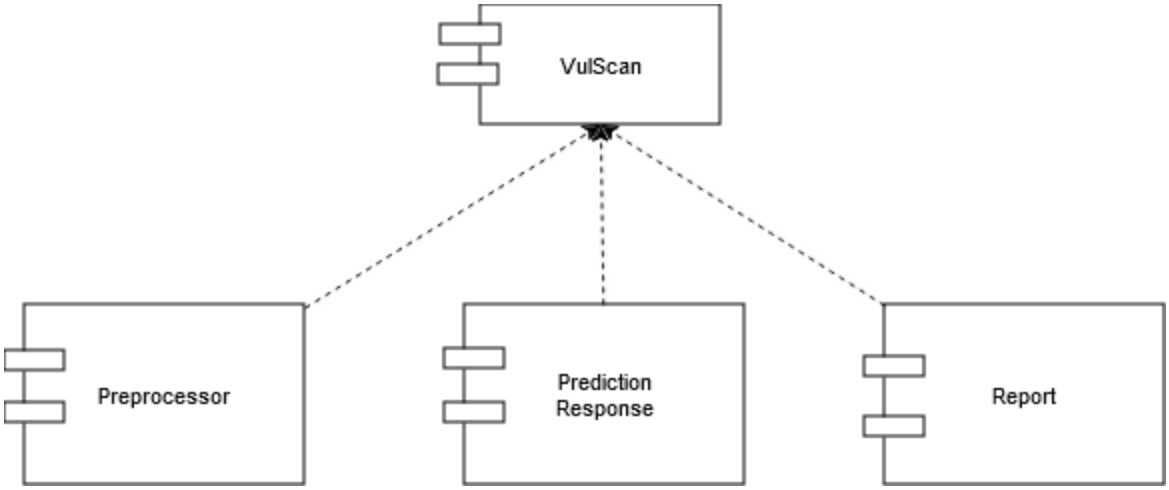


Figure 10: Top level components

5.4 Instantiation of the Architecture

The components identified in Section - 5.3 are broad. So they are refined and instantiated into smaller components. We use different architectural patterns which better suite each component

Component: PreProcessor

Pattern: Pipe and Filter

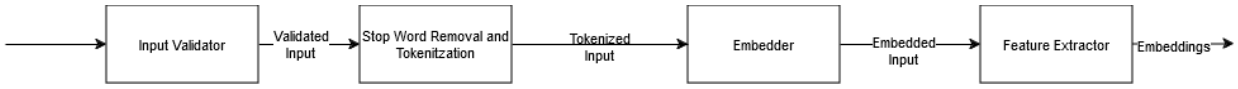


Figure 11: Pipe and Filter Architecture for PreProcessor

The PreProcessor component processes the code to generate query and code corpus. Pipe and Filter architecture is used so that the input can be processed sequentially through a set of defined steps.

Component: PredictionResponse

Pattern: Pipe and Filter

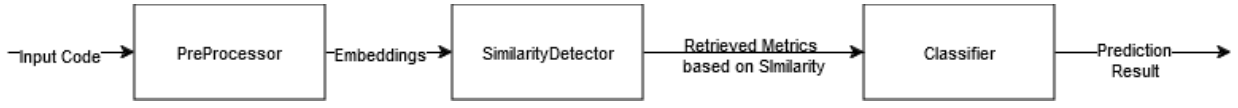


Figure 12: Pipe and Filter of PredictionResponse

In the PredictionResponse component the output from the previous step is sent through a set of pipes and filters and the prediction is obtained through the use of classifier. Here the pipes are reusable and scalable.

Component: Report

Pattern: Model-View-Controller (MVC)

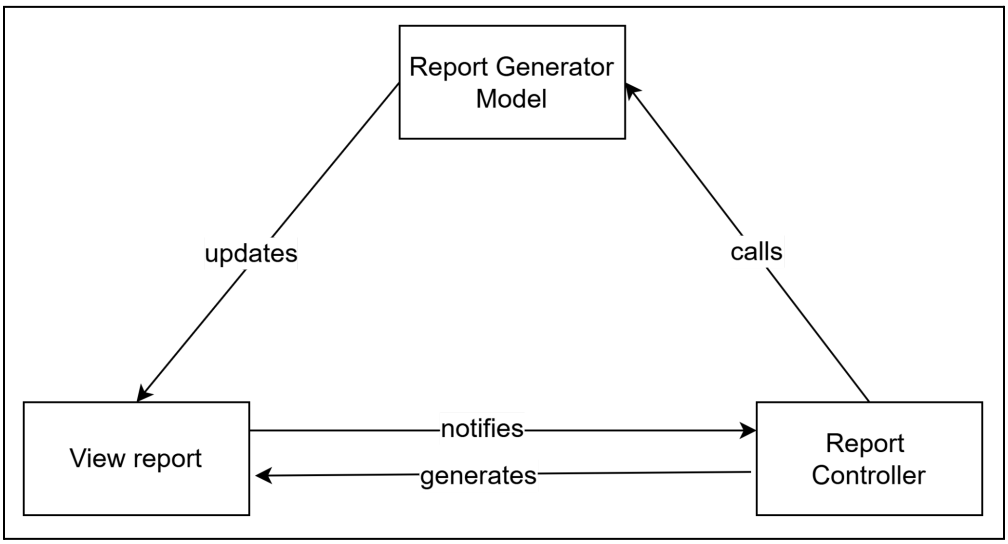


Figure 13: MVC of Report

The Github Repository manager is best suited for MVC patterns. Here, the issues are placed and the ranked list is shown using view. The model handles the new issue and updates the view with the new ranked list. The controller facilitates communication between model and view.

6. Preliminary Test Plan

In this chapter, a high level description of testing goals and summary of features to be tested are presented.

6.1 High Level Description of Testing Goals

Testing goals for this tool is represented below from a high level-

- To demonstrate that the tool meets its requirements
- To ensure that the input codebase is a proper Github repository
- To ensure input is parsed properly
- To ensure that it produces appropriate code corpus and code hunks
- To ensure REST API calls are handled properly
- To ensure that the system properly gives output according to the required format
- To ensure that the system doesn't cause any memory or storage overflow

- To ensure that the tool runs smoothly
- To ensure that the system outputs the correct ranked list of files.

6.2 Test Cases

Test cases for the testing of this tool are documented below-

Test ID: T1

Test Case: Test if the system can successfully extract functions from a source code file.

Input Test Data: A zip file containing a C project with multiple .c and .h files.

Steps To Be Executed:

1. Upload the zip file to the system.
2. Trigger the function extraction process.
3. Verify the extracted functions list in the output.

Expected Result: All functions in the provided C files are extracted and stored for further processing.

Target Item: CodePreProcessor.extractFunctions()

Pass/Fail: Pending

Test ID: T2

Test Case: Test if metrics are correctly extracted for a given function.

Input Test Data: A C function

Steps To Be Executed:

1. Provide the sampleFunction as input.
2. Trigger the metrics extraction process.
3. Check the extracted metrics (e.g., availability impact, access complexity).

Expected Result: Metrics are extracted and returned as a structured output (e.g., JSON with all metric values).

Target Item: MetricsExtractor.extractMetrics()

Pass/Fail: Pending

Test ID: T3

Test Case: Test if the system can predict vulnerabilities for a function.

Input Test Data:

- Function: sampleFunction (from Test Case 2).
- Metrics: {availability impact: High, access complexity: Low, confidentiality impact: High}.

Steps To Be Executed:

1. Pass the function and metrics to the VulnerabilityPredictor.predictVulnerability() method.
2. Trigger the vulnerability prediction process.
3. Verify the prediction result (e.g., vulnerable/not vulnerable)

Expected Result: The system predicts the function as vulnerable and classifies it correctly.

Target Item: VulnerabilityPredictor.predictVulnerability()

Pass/Fail: Pending

Test ID: T4

Test Case: Test if the system incorporates user feedback correctly.

Input Test Data:

- Feedback: A function marked as "not vulnerable" is flagged as a false negative.
- Function: sampleFunction (from Test Case 2).

Steps To Be Executed:

1. Submit feedback for a false negative prediction.
2. Trigger the feedback processing system.
3. Verify that the system logs the feedback and adjusts its prediction mechanism.

Expected Result: Feedback is logged and used for improving prediction accuracy.

Target Item: FeedbackProcessor.processFeedback()

Pass/Fail: Pending

Test ID: T5

Test Case: Test if the RAG pipeline retrieves relevant entries for a given query.

Input Test Data: Query

Steps To Be Executed:

1. Call the LLMProcessor.queryKnowledgeBase() method with the input query.
2. Verify that the retrieved entries are relevant to the authentication-related function.

   Expected Result: The RAG pipeline retrieves entries similar to the input query based on vector similarity.

   Target Item: LLMProcessor.queryKnowledgeBase()

   Pass/Fail: Pending

Test ID: T6

Test Case: Test if a report is generated correctly after processing user input.
Input Test Data:

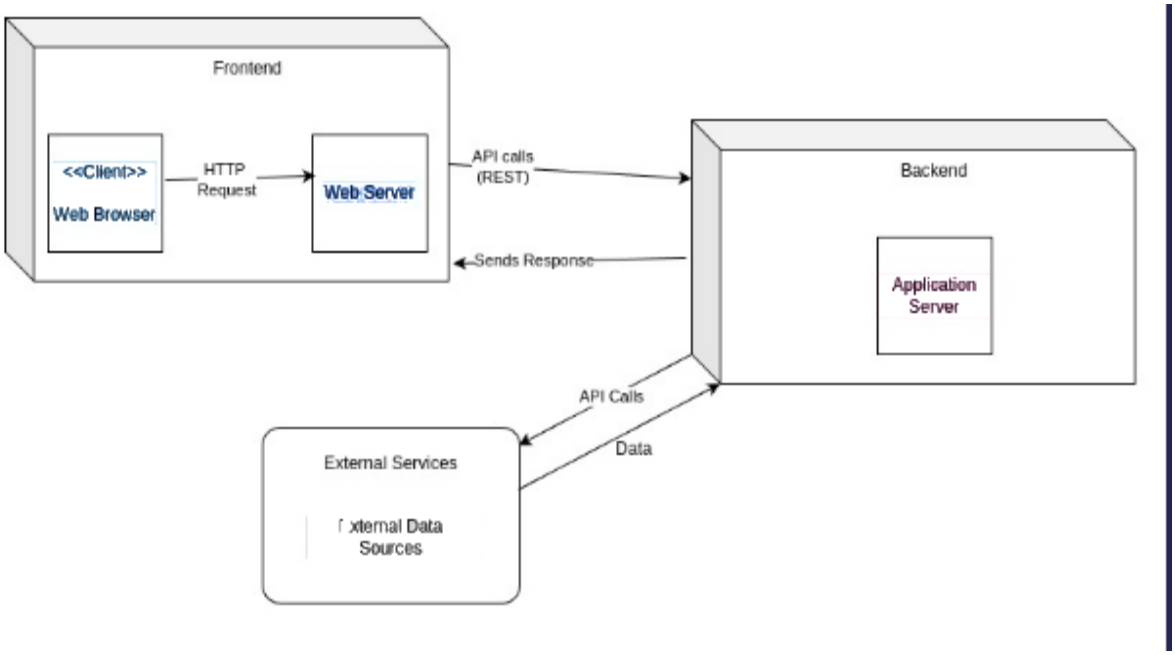
- Input Function:
- **Prediction Result: {status: vulnerable, type: buffer overflow}.**

Steps To Be Executed:

1. Provide the input function and its prediction result to the ReportGenerator.createReport() method.
2. Trigger the report generation process.
3. Verify the content and format of the generated PDF.

Expected Result: A PDF report containing the function, its metrics, and vulnerability details is generated successfully.
Target Item: ReportGenerator.createReport()
Pass/Fail: Pending

7. Deployment



9. Conclusion

VulScan is an innovative and developer-centric tool designed to revolutionize the process of fault localization. By using advanced techniques, VulScan not only assists developers in identifying vulnerabilities but also provides actionable insights for remediation. This project demonstrates how cutting-edge technology can address critical security challenges in a practical, efficient, and user-focused manner.